Microsoft Interview Preparation: 7-Day Project Revision Roadmap

This roadmap is designed for 7 days of revision (3 hours daily) to prepare you to defend PathFinder (Smart Trip Route Optimizer) in your Microsoft interview. It focuses on the key areas Microsoft interviewers care about: Data Structures & Algorithms (DSA), System Design & Scalability, Resilient API Integration, Security, and Frontend/UX Architecture.

Roadmap Overview

Day 1: Algorithmic Core - Held-Karp Algorithm & Bitmask DP

- Daily Goal: Master the Traveling Salesperson Problem (TSP) mathematical modeling and exact Dynamic Programming solution.
- Target Code: tspSolver.js

Key Concepts to Master:

1. The Traveling Salesperson Problem (TSP):
- Why is it NP-Hard? (No polynomial-time algorithm exists; checking all permutations is brute-force $O(N!)$).
- What makes it asymmetric in real life? (One-way streets, traffic, slope: $cost(A \to B) \neq cost(B \to A)$).
2. Held-Karp State Transition:
- The state is defined as dp[mask][u]: the minimum cost to visit all nodes in the subset represented by mask ending at node u.
- Transition Equation:

\[

dp[mask][v] = \min_{u \in mask} (dp[mask \setminus \{v\}][u] + cost(u, v))

\]

- Bitwise operations used:
- mask & (1 << u): Check if node u is present in the current subset.
- mask | (1 << v): Add node v to the current subset.
- mask ^ (1 << currNode): Remove currNode from the subset (used during path reconstruction).
3. Complexity Analysis:
- Time Complexity: $O(N^2 2^N)$ because there are 2^N subsets (masks), N ending states, and we iterate over N possible previous states.
- Space Complexity: $O(N 2^N)$ to store the DP and parent tracking tables.

NOTE: [!NOTE] Be prepared to draw a simple transition graph for $N=3$ or $N=4$ nodes showing how state is built up.

Day 2: Mapping, OSRM Matrix, and Fallback Math

- Daily Goal: Master how spatial routing works, external API integration, and mathematical coordinate estimations.
- Target Code: osrmService.js

Key Concepts to Master:

1. OSRM Table API:
- Understand how OSRM (Open Source Routing Machine) constructs distance matrices from OpenStreetMap street

networks using Contraction Hierarchies.

- Profile configurations: foot vs bike vs driving (average velocities, road segment permissions).

2. Haversine Formula:

- Formula for great-circle distance between coordinates on a sphere:

$$
a = \sin^2\left(\frac{\Delta \phi}{2}\right) + \cos(\phi_1)\cos(\phi_2)\sin^2\left(\frac{\Delta \lambda}{2}\right)
$$

$$
c = 2 \cdot \arctan2(\sqrt{a}, \sqrt{1 - a}), \quad d = R \cdot c
$$

- Why do we use it? As a spherical approximation of Earth's surface (radius $R \approx 6,371 \text{ km}$).

3. Resilient Fallback Pattern:

- Backend Fallback: If the OSRM server times out (4000ms), the backend calculates a Haversine distance matrix and divides it by speed values per mode (getSpeedForMode) to approximate duration.
- Frontend Fallback: If the backend is completely offline, the client uses its own haversineDistance helper.

Day 3: Backend Architecture, MVC, & Robust Error Handling

- Daily Goal: Understand the structure of an enterprise-grade Express.js application and the ES Modules system.
- Target Code: app.js, index.js, and async-handler.js

Key Concepts to Master:

1. Server vs. App Separation:

- Why is app.js (middleware + routes) separated from index.js (database connection + .listen())? (Enables serverless deployment and isolated integration testing using Supertest without opening active network ports).

2. ES Modules Configuration:

- "type": "module" in package.json.
- Difference between ESM (import/export) and CommonJS (require/module.exports). (ESM supports static analysis, tree-shaking, and asynchronous top-level imports).

3. Centralized Error Handling:

- How asyncHandler acts as a wrapper to catch rejected promises in async route handlers and automatically pass them to the global Express error middleware (next(err)).
- Standardized JSON responses using custom ApiError and ApiResponse utilities.

Day 4: Security, Authentication, & JWT Systems

- Daily Goal: Master user session management, credentials security, and token safety.
- Target Code: backend/src/routes/auth.routes.js, backend/src/middleware/auth.middleware.js

Key Concepts to Master:

1. Double Token JWT Architecture:

- Access Token: Short-lived (e.g., 15 mins), sent in requests to verify identity.
- Refresh Token: Long-lived (e.g., 10 days), stored in the database, used to request new access tokens when they expire.

2. Security Vulnerabilities & Mitigations:

- XSS (Cross-Site Scripting): If tokens are stored in localStorage, malicious JS can steal them. Mitigated by storing tokens in HTTP-only, Secure, SameSite=Strict cookies.
- CSRF (Cross-Site Request Forgery): Mitigated by checking Custom Headers (like Authorization bearer token) and setting strict CORS origins instead of wildcard * in production.
- Password Hashing: Always salt and hash passwords using bcrypt before database persistence.

Day 5: Frontend State, Hash Routing, & Map Integration

- Daily Goal: Understand the React UI flow, context-driven state, and interactive mapping.
- Target Code: App.jsx, RouteContext.jsx, PlaceSelect.jsx

Key Concepts to Master:

1. State Hydration and Context:

- How RouteProvider shares active navigation, selected place IDs, mode, and results across all pages.
- State isolation: Resetting routes triggers re-optimization without clearing the current city.

2. Hash-Based Client Routing:

- Using the hashchange browser event listener to sync React state (landing -> select -> results) with URL hashes (#select, #results).
- Why use hash routing over a library like React Router? (Simpler setup, works out-of-the-box on static hosting platforms without server rewrites).

3. Interactive Maps (Leaflet & React-Leaflet):

- Rendering polylines from GeoJSON coordinates.
- Custom icons and dynamic zoom centering (selectedCity.center and selectedCity.zoom).
- Synchronicity: Hovering a route itinerary spot triggers highlighting on the map markers.

Day 6: System Design, Scalability, & Large Scale Heuristics

- Daily Goal: Answer the crucial question: *"How would you design and scale this system to support thousands of locations and millions of daily users?"*

Core Architectural Tradeoffs:

1. Solving TSP for Large N:

- Held-Karp ($O(N^2 2^N)$) hits a resource wall at $N > 15-20$.
- Nearest Neighbor (NN): Start at start node, always visit the closest unvisited node. Time complexity: $O(N^2)$, space complexity: $O(N)$. Highly efficient but can produce paths 20-30% longer than optimal.
- 2-Opt Local Search: Starts with a random/NN path and iteratively swap edges to eliminate overlaps. Time complexity: $O(N^2)$ per iteration.
- Genetic Algorithms & Simulated Annealing: Metaheuristics for near-optimal routes on very large networks ($N \ge 1000$).

2. Scaling the Backend:

- Redis Caching: Cache OSRM tables using hashed geo-coordinate combinations. If another user requests the same locations, return from cache ($O(1)$) instead of fetching OSRM.
- Event Loop Protection: Node.js is single-threaded. Running Held-Karp for $N=15$ blocks incoming requests.
- Solution: Offload TSP computations to a Worker Thread Pool (worker_threads module) or a background queue worker system (e.g., BullMQ / Redis + Node.js cluster).

Day 7: Mock Interview & Project Walkthrough Practice

- Daily Goal: Practice presenting the project clearly, explaining tradeoffs, and highlighting accomplishments.

3-Step Project Pitch Strategy

1. The Hook (Situation & Task):
- *"I built PathFinder, a travel optimization web app that solves the Traveling Salesperson Problem for city tourism. The system takes multiple user sightseeing choices and arranges them into the mathematically shortest itinerary using real-world road networks rather than straight-line distance."*
2. The Core Technical Decisions (Action):
- *"For exact optimization up to 12 locations, I implemented the Held-Karp Bitmask Dynamic Programming algorithm. I integrated the Open Source Routing Machine (OSRM) to query road travel times and distance matrices. To make the application highly resilient, I engineered backend and frontend fallback layers using Haversine formulas and a Nearest Neighbor heuristic, ensuring the app works even when external routing servers go offline."*
3. The Results (Result):
- *"The system delivers exact optimization in milliseconds, integrates real-time Leaflet maps, and includes security layers such as JWT token rotation stored in HTTP-only cookies to protect user accounts."*

Top 5 Microsoft Interview Trap Questions:

- Q1: *"Why is exact TSP calculation limited to 12 places in your UI?"*
- Answer: Held-Karp has exponential time complexity $O(N^2 2^N)$. For $N=12$, it takes $\approx 49,152$ operations. For $N=20$, it takes $\approx 2 \times 10^7$ operations. Limiting to 12 ensures millisecond response times and prevents blocking the Node.js single-threaded event loop.
- Q2: *"Why didn't you store your JWT in localStorage?"*
- Answer: localStorage is vulnerable to Cross-Site Scripting (XSS) attacks. If an attacker injects a script, they can steal the tokens. Storing them in HTTP-only cookies makes them inaccessible to JavaScript.
- Q3: *"What happens if the OSRM API returns an error or is slow?"*
- Answer: The service automatically catches the error or handles timeouts, falling back to a Haversine formula calculation over coordinates with static velocity mappings.
- Q4: *"How would you improve map rendering performance if there were 10,000 points?"*
- Answer: Use Leaflet Marker Clustering, Canvas-based rendering instead of SVG/DOM path rendering, and only render elements visible in the current map viewport.
- Q5: *"How would you test this application?"*
- Answer: Unit test the Held-Karp TSP solver logic with mock matrices, integration test the REST API endpoints using Supertest, and run E2E UI testing on user route flow using Playwright/Cypress.

Day 1 Revision: Held-Karp Algorithm & Bitmask DP (3 Hours)

Today's focus is the algorithmic foundation of the PathFinder engine. You must be able to write, trace, and analyze the Held-Karp Dynamic Programming algorithm on a whiteboard.

Hourly Breakdown

```
[ Hour 1 ]  Math & Bitmask Foundations (Set representation)
[ Hour 2 ]  Code Execution & State Transitions Walkthrough
[ Hour 3 ]  Interview Q&A, Edge Cases, & Time/Space Tradeoffs
```

Hour 1: Bitmask & DP State Foundations

In the Traveling Salesperson Problem (TSP), a brute-force search evaluates all $N!$ permutations, which becomes impossible for $N > 10$. The Held-Karp algorithm reduces this to $O(N^2 2^N)$ using Dynamic Programming.

1. Understanding the State: `dp[mask][u]`

- mask (Integer): A bitmask representing the subset of visited vertices.
- If $N = 4$, our vertices are $\{0, 1, 2, 3\}$.
- A mask of 13 (binary 1101) means vertices 0, 2, and 3 have been visited (reading right to left: $2^0 + 2^2 + 2^3 = 1 + 4 + 8 = 13$).
- u (Integer): The current/last visited vertex in this subset.
- dp[mask][u] stores: The minimum distance to start at vertex 0, visit all vertices set in the mask, and end at vertex u.

2. Crucial Bitwise Tools:

- Shift (1 << u): Generates a mask with only the u-th bit set. E.g., 1 << 2 is 0100 (binary) or 4.
- Set Intersection (mask & (1 << u)): Checks if vertex u is in the subset. Returns non-zero (truthy) if the bit is set.
- Set Union (mask | (1 << v)): Inserts vertex v into the visited set.
- Set Difference (mask ^ (1 << u)): Toggles the u-th bit off (used during backtracking to reconstruct the path).

Hour 2: Transition Walkthrough & Code Dry Run

Let's dissect the main loop from tspSolver.js:

```
for (let mask = 1; mask < MAX_MASK; mask++) {
  for (let u = 0; u < N; u++) {
    // 1. If 'u' is not in the mask, or this state is unreachable, skip
    if (!(mask & (1 << u)) || dp[mask][u] === Infinity) continue;

    // 2. Try transitioning to an unvisited vertex 'v'
    for (let v = 0; v < N; v++) {
      if (mask & (1 << v)) continue; // 'v' is already visited

      const nextMask = mask | (1 << v);
      const newDist = dp[mask][u] + matrix[u][v];

      // 3. Relax the state: keep the minimum distance
      if (newDist < dp[nextMask][v]) {
        dp[nextMask][v] = newDist;
        parent[nextMask][v] = u; // Track parent for backtracking
```

```
        }
      }
    }
}
```

Step-by-Step Dry Run (N=3)

Suppose we have a 3-node graph with the following distance matrix:

- matrix[0][1] = 10, matrix[0][2] = 15
- matrix[1][0] = 10, matrix[1][2] = 20
- matrix[2][0] = 15, matrix[2][1] = 20

****Step 1: Initialization****

- MAX_MASK = 1 << 3 = 8 (masks from 0 to 7).
- Base case: Start at node 0.
- mask = 1 (001 in binary: only node 0 visited).
- dp[1][0] = 0. All other states are Infinity.

****Step 2: Mask Iterations****

- mask = 1 (001 - visited: {0}):
- u = 0: Reachable.
- Check transitions to unvisited v = 1:
- nextMask = 1 | (1 << 1) = 3 (011 - visited: {0, 1}).
- newDist = dp[1][0] + matrix[0][1] = 0 + 10 = 10.
- dp[3][1] = 10, parent[3][1] = 0.
- Check transitions to unvisited v = 2:
- nextMask = 1 | (1 << 2) = 5 (101 - visited: {0, 2}).
- newDist = dp[1][0] + matrix[0][2] = 0 + 15 = 15.
- dp[5][2] = 15, parent[5][2] = 0.
- mask = 3 (011 - visited: {0, 1}):
- u = 1: Reachable (dp[3][1] = 10).
- Check transitions to unvisited v = 2:
- nextMask = 3 | (1 << 2) = 7 (111 - visited: {0, 1, 2}).
- newDist = dp[3][1] + matrix[1][2] = 10 + 20 = 30.
- dp[7][2] = 30, parent[7][2] = 1.
- mask = 5 (101 - visited: {0, 2}):
- u = 2: Reachable (dp[5][2] = 15).
- Check transitions to unvisited v = 1:
- nextMask = 5 | (1 << 1) = 7 (111 - visited: {0, 1, 2}).
- newDist = dp[5][2] + matrix[2][1] = 15 + 20 = 35.
- Since 30 < 35, we do not update dp[7][1].

****Step 3: Finding the Minimum Path & Backtracking****

- fullMask = 7 (111).
- One-Way Path (roundTrip = false):
- We scan dp[7][u] for all nodes u.

- dp[7][1] = Infinity (cannot end at 1 because we went $0 \to 2 \to 1$, but we didn't update it since it was worse).
- dp[7][2] = 30 (path: $0 \to 1 \to 2$).
- Minimum cost is 30, ending at lastNode = 2.
- Path Reconstruction:
- Begin with currMask = 7, currNode = 2.
- Push 2 to path.
- Find parent: prevNode = parent[7][2] = 1.
- Update mask: currMask = 7 ^ (1 << 2) = 3 (011).
- Move to next: currNode = 1. Push 1 to path.
- Find parent: prevNode = parent[3][1] = 0.
- Update mask: currMask = 3 ^ (1 << 1) = 1 (001).
- Move to next: currNode = 0. Push 0 to path.
- parent[1][0] = -1. Loop terminates.
- Reverse path: [0, 1, 2].

Hour 3: Microsoft Interview Questions & Answers

Be prepared to answer these exact questions on Day 1:

Q1: What are the time and space complexities of this algorithm, and why?

- Time Complexity: $O(N^2 2^N)$. There are 2^N subsets. For each subset, we iterate through N possible ending nodes (u), and try to transition to N possible next nodes (v).
- Space Complexity: $O(N 2^N)$. We store tables dp, durDp, and parent of size $2^N \times N$.

Q2: Why does the code start `mask = 1` and `dp[1][0] = 0`? What assumption does this make?

- Answer: It assumes the tour always starts at node index 0 (represented by bit index 0). If the user selects a specific start node, we pre-process the array to swap that start node to index 0 (which is done in optimizeRoute at tspSolver.js).

Q3: What happens if `roundTrip` is set to `true` vs `false`?

- Answer:
- If roundTrip is true, we find the path that visits all vertices and returns to the origin: we minimize dp[fullMask][u] + matrix[u][0] (adds the cost of traveling back to node 0 from the last node u).
- If roundTrip is false, we find the shortest open path (Hamiltonian path) visiting all nodes: we take the minimum of dp[fullMask][u] for any final node u.

Q4: Can we optimize the space complexity?

- Answer: Not easily for the full path reconstruction. If we only need the *minimum distance* (and not the actual path sequence), we could optimize it because computing DP values for mask of size k only depends on mask values of size $k-1$. However, because we must reconstruct the optimal route, we need the full parent table, meaning $O(N 2^N)$ space is required.

Day 1 Action Item

Open a code editor or write on paper:

1. Try to write the heldKarpTSP function from memory.
2. Explain the bitwise line currMask = currMask ^ (1 << currNode) out loud as if explaining to an interviewer.

JWT Authentication Implementation & Code Walkthrough

This document contains a complete walk-through of the JSON Web Token (JWT) authentication system implemented in the PathFinder project.

Folder Structure Overview

The authentication logic is contained within the following directory:

backend/src/

- models/user.models.js - Schema, password hashing, and token signature declarations.
- controllers/auth.controller.js - Controller logic for registration, login, logout, and token refresh.
- middleware/auth.middleware.js - Request interceptor to verify tokens.
- routes/auth.routes.js - Route mapping mounting the controller functions and middlewares.

Step-by-Step Code Walkthrough

Step 1: Model Declaration & Token Signing Methods

File: backend/src/models/user.models.js

1. Password Hashing Middleware:

Before a user document is saved to MongoDB, a pre-save hook intercepts it to hash the plain-text password using bcrypt:

```
userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

2. Access Token Generation (generateAccessToken):

Generates a token carrying payload details signed with process.env.ACCESS_TOKEN_SECRET:

```
userSchema.methods.generateAccessToken = function () {
  return jwt.sign(
    { _id: this._id, email: this.email, role: this.role, username: this.username },
    process.env.ACCESS_TOKEN_SECRET,
    { expiresIn: process.env.ACCESS_TOKEN_EXPIRY || "15m" }
  );
};
```

3. Refresh Token Generation (generateRefreshToken):

Generates a long-lived session signature:

```
userSchema.methods.generateRefreshToken = function () {
  return jwt.sign(
    { _id: this._id },
    process.env.REFRESH_TOKEN_SECRET,
    { expiresIn: process.env.REFRESH_TOKEN_EXPIRY || "7d" }
  );
};
```

Step 2: Generating & Delivering Tokens (Login)

File: backend/src/controllers/auth.controller.js

1. Token Generation Orchestrator (generateAccessAndRefreshTokens):

Signs both tokens and saves the refresh token string directly into the database on the user's document for session tracking:

```
const generateAccessAndRefreshTokens = async (userId) => {
  const user = await User.findById(userId);
  const accessToken = user.generateAccessToken();
  const refreshToken = user.generateRefreshToken();
  user.refreshToken = refreshToken;
  await user.save({ validateBeforeSave: false });
  return { accessToken, refreshToken };
};
```

2. Login Controller (login):

Checks credentials, initiates token generation, sets the HTTP-Only cookies, and returns the response:

```
const { accessToken, refreshToken } = await generateAccessAndRefreshTokens(user._id);

const cookieOptions = {
  httpOnly: true,
  secure: process.env.NODE_ENV === "production",
  sameSite: process.env.NODE_ENV === "production" ? "none" : "lax",
};

return res
  .status(200)
  .cookie("accessToken", accessToken, cookieOptions)
  .cookie("refreshToken", refreshToken, cookieOptions)
    .json(new ApiResponse(200, { user: loggedInUser, accessToken, refreshToken }, "User logged in
successfully"));
```

Step 3: Protecting Endpoints with Verification Middleware

File: backend/src/middleware/auth.middleware.js

The verifyJWT middleware intercepts requests to private/protected endpoints:

1. Extraction: Looks for the token inside incoming cookies first, and falls back to check the Authorization header:

```
const token = req.cookies?.accessToken || req.header("Authorization")?.replace("Bearer ", "");
```

2. Verification: Validates the signature using ACCESS_TOKEN_SECRET. If valid, it fetches the user object from MongoDB, excludes sensitive data, and mounts it onto req.user:

```
const decodedToken = jwt.verify(token, process.env.ACCESS_TOKEN_SECRET);
const user = await User.findById(decodedToken?._id).select("-password -refreshToken");
req.user = user;
next(); // Pass request control to the next controller/handler
```

Step 4: Token Rotation (Refresh Endpoint)

File: backend/src/controllers/auth.controller.js

The refreshAccessToken function takes a refresh token, verifies it, updates the database, and issues new cookies:

```
const decodedToken = jwt.verify(incomingRefreshToken, process.env.REFRESH_TOKEN_SECRET);
const user = await User.findById(decodedToken._id);
if (incomingRefreshToken !== user.refreshToken) throw new ApiError(401, "Refresh token expired");

// Issue and save new tokens (Token Rotation pattern)
const { accessToken, refreshToken } = await generateAccessAndRefreshTokens(user._id);
```

Step 5: Logging Out

File: backend/src/controllers/auth.controller.js

The logoutUser controller invalidates the session:

1. Clears the refreshToken inside the user's database document.
2. Directs the client to delete both browser cookies via clearCookie:

```
  return res
    .status(200)
    .clearCookie("accessToken", cookieOptions)
    .clearCookie("refreshToken", cookieOptions)
    .json(new ApiResponse(200, {}, "User logged out successfully"));
```

Microsoft Interview Focus: Core Concepts & Defenses

1. XSS (Cross-Site Scripting) Defense:

Storing tokens in localStorage makes them accessible to malicious scripts. Storing them in cookies with httpOnly: true prevents javascript from reading them.

2. CSRF (Cross-Site Request Forgery) Defense:

Using sameSite: "strict" or sameSite: "lax" ensures cookies are not sent with cross-site requests. Checking custom request headers (like Authorization) further secures APIs since cookies alone cannot authenticate actions.

3. Token Rotation / Dual Token Security:

Short-lived access tokens limit the window of opportunity if a token is intercepted. Refresh tokens are stored securely in a database and can be revoked instantly at logout or password changes, closing active sessions immediately.